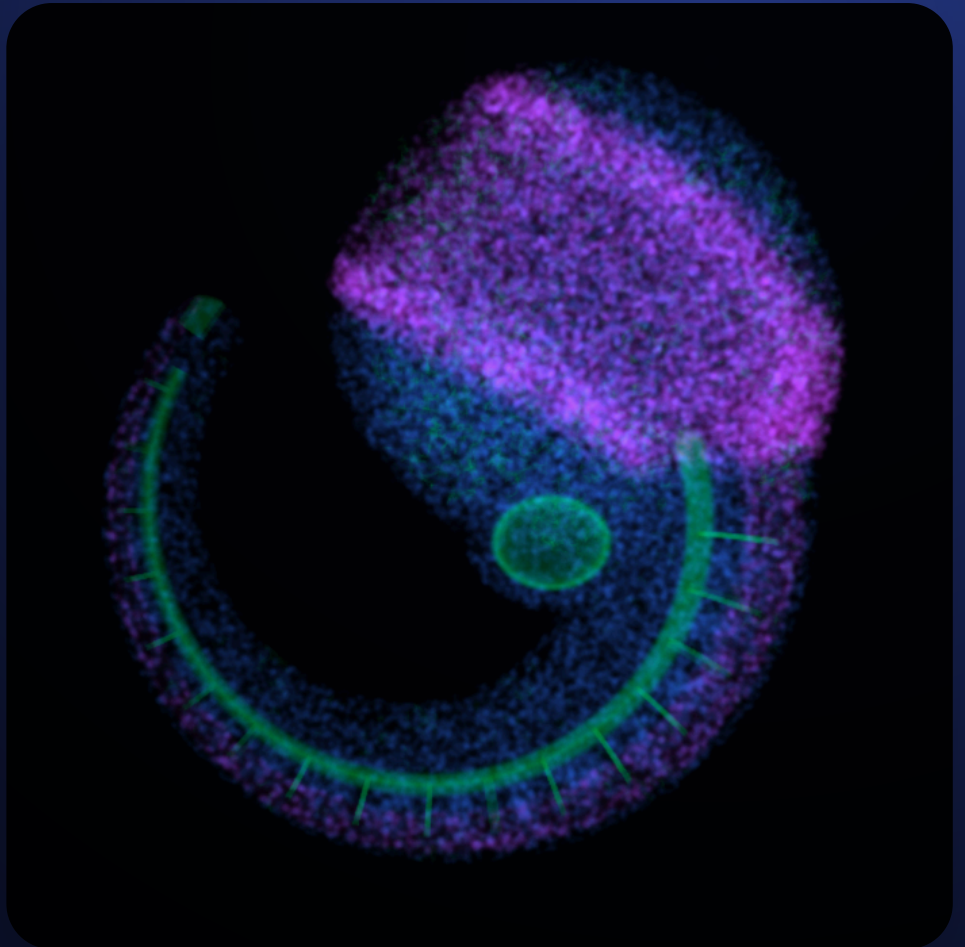




IRIBHM · ULB — LUMEN3D

# Lumen3D — the essentials

What happens to your Imaris file, from the microscope to the screen — and how to make good use of it

---

The biologist's guide: prepare an acquisition, have it processed, put it online, explore it in 3D, measure, build figures, and know how to interpret what you see. Each section points to the chapter of the full documentation ("Lumen3D, from A to Z") that gives every detail.

# Contents

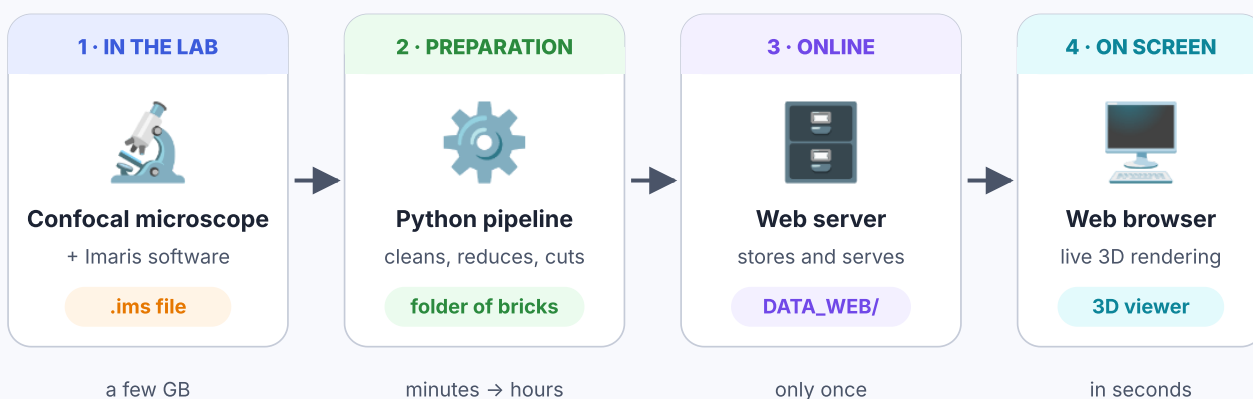
|                                                         |  |
|---------------------------------------------------------|--|
| <b>1. Lumen3D in two pages</b>                          |  |
| 1.1 Why a preparation step is essential                 |  |
| 1.2 Three kinds of datasets                             |  |
| <b>2. Before the pipeline: prepare your acquisition</b> |  |
| 2.1 The file name: stage and embryo                     |  |
| 2.2 What Imaris must contain                            |  |
| 2.3 Colours, channels, 8- or 16-bit data                |  |
| 2.4 Timelapse with cell tracking                        |  |
| <b>3. Running the pipeline</b>                          |  |
| <b>4. Cleaning the image</b>                            |  |
| 4.1 Floor and ceiling                                   |  |
| 4.2 The mask: protecting the real signal                |  |
| 4.3 Conversion to 8 bits                                |  |
| 4.4 For a timelapse: one shared window                  |  |
| <b>5. Reducing the size without loss</b>                |  |
| 5.1 The pyramid                                         |  |
| 5.2 Lossless compression                                |  |
| 5.3 Empty bricks are not stored                         |  |
| <b>6. Bricks and formats</b>                            |  |
| <b>7. The end of the pipeline</b>                       |  |
| <b>8. Putting the dataset online and setting it up</b>  |  |
| 8.1 The settings that matter to you                     |  |
| <b>9. Exploring in 3D</b>                               |  |
| 9.1 Quality and loading                                 |  |
| 9.2 How the image is computed                           |  |
| 9.3 The three render modes                              |  |
| <b>10. Adjusting how a channel is displayed</b>         |  |
| <b>11. Measuring and exploring the inside</b>           |  |
| <b>12. Making figures and exporting</b>                 |  |
| <b>13. Timelapses and cell tracking</b>                 |  |
| <b>14. 2D photographs</b>                               |  |
| <b>15. Interpreting what you see correctly</b>          |  |
| 15.1 What the platform does not do                      |  |
| <b>16. What happens if... ?</b>                         |  |
| <b>17. Words to know</b>                                |  |

# 1. Lumen3D in two pages

## 🕒 IN 30 SECONDS

- Lumen3D shows confocal volumes of **several gigabytes** in a plain web browser, with nothing to install.
- A **pipeline** prepares each Imaris file **once**; the original file is never modified.
- The **viewer** downloads only what the screen needs, then computes the 3D image on the graphics card.

## The journey of an acquisition: from the microscope to the screen



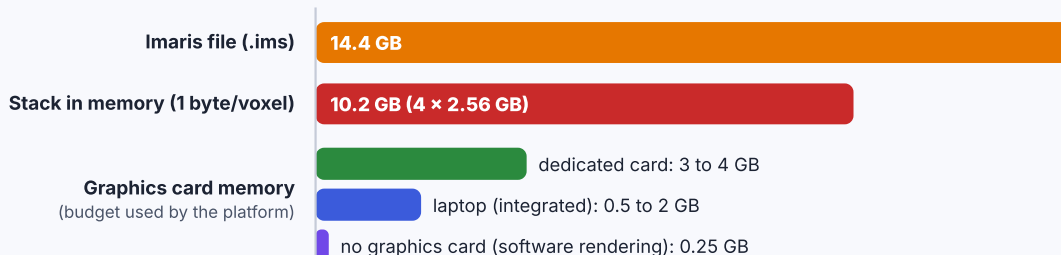
*The original file is never modified: the pipeline makes a web-optimised copy of it.*

▲ From the microscope to the screen, in four steps.

## 1.1 Why a preparation step is essential

### Why a browser cannot simply “open” the stack

Real example: embryo of  $3789 \times 3789 \times 178$  voxels, 4 channels (the pipeline's reference dataset)

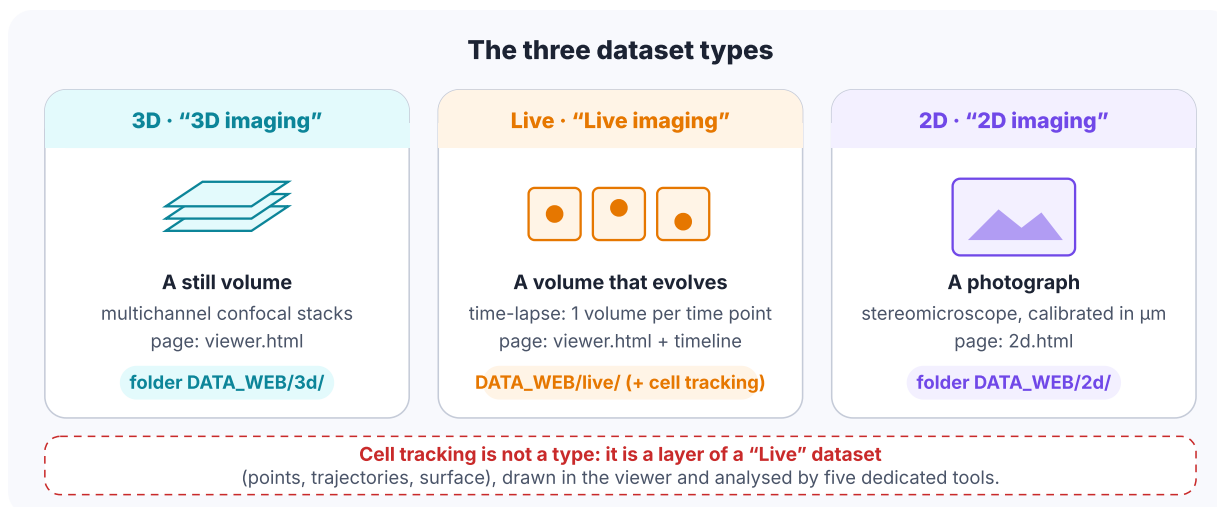


**Conclusion: we must cut up, shrink, and load only what the screen shows.**

At ~5 MB/s (speed measured on a host), downloading 10 GB in one go would take over half an hour.

▲ Even reduced to 1 byte per voxel, a large embryo exceeds the memory of an ordinary computer: it has to be cut up, and only the useful part loaded.

## 1.2 Three kinds of datasets



▲ Three types, three display pages. The displayed names (3D, 2D, Live) can be renamed by the administrator.

| TYPE | WHAT IT IS                                   | PAGE                 | SPECIAL FEATURE                  |
|------|----------------------------------------------|----------------------|----------------------------------|
| 3D   | a confocal stack, up to 4 channels displayed | 3D viewer            | measuring tools, slices, Studio  |
| Live | a time series (timelapse)                    | 3D viewer + timeline | can carry a <b>cell tracking</b> |
| 2D   | a calibrated photograph (stereomicroscope)   | 2D page              | X-gal staining isolation         |

### GOOD TO KNOW

This document summarises the **full documentation** ("Lumen3D, from A to Z", 21 chapters). The pointers → **ch. N** lead to the chapter that explains everything in detail. The volume images come from **synthetic demonstration embryos**, processed by the real pipeline.

## 2. Before the pipeline: prepare your acquisition

### IN 30 SECONDS

- The **file name** gives the stage and the embryo number: choose it carefully.
- The **calibration** (voxel size) and the **channel names** are read from the Imaris file: check them before exporting.
- For a tracked timelapse, prepare the **tracking source** (one of the three accepted formats).

## 2.1 The file name: stage and embryo

The pipeline reads the stage ( `E...` ) and the embryo ( `Em...` ) from the name. The exact rules:

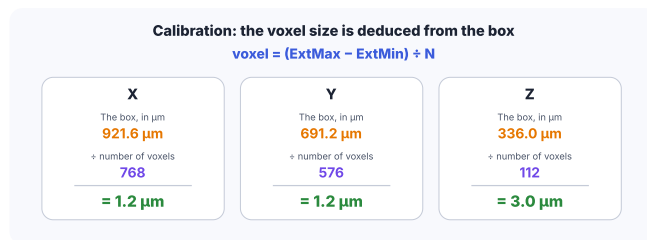
| IN THE NAME                                                                                       | STAGE READ          | EMBRYO READ |
|---------------------------------------------------------------------------------------------------|---------------------|-------------|
| <code>...-E8.5-Em1-...</code> or <code>...-E8,5-...</code>                                        | E8.5                | Em1         |
| <code>...-E8-5-...</code> (dash: 25, 5 or 75 only)                                                | E8.5                | —           |
| <code>...-E85-...</code> · <code>...-E105-...</code> · <code>...-E95-...</code><br>(compact form) | E8.5 · E10.5 · E9.5 | —           |
| <code>...-E8-1-DAPI...</code> (the "1" is not a fraction)                                         | E8                  | Em1         |
| <code>...-E95-2-...</code>                                                                        | E9.5                | Em2         |
| no <code>E...</code> recognised                                                                   | "Unknown"           | —           |

The published folder name is the file name, cleaned (characters other than letters, digits, `.`, `_`, `-` replaced by `-`). All of this can be corrected later in the administration. → ch. 8

## 2.2 What Imaris must contain

Check in Imaris before exporting:

- **the calibration** (voxel size in X, Y, Z): without it, no scale bar and no measurement in  $\mu\text{m}$ ;
- **the name of each channel** (DAPI, Pecam1...): an empty name becomes "Channel 1", "Channel 2"...;
- for a timelapse, **the time** of each frame: the interval kept is the median of the gaps.



▲ The voxel size is computed: extent  $\div$  number of voxels, axis by axis. Here  $921.6 \mu\text{m} \div 768 = 1.2 \mu\text{m}$ .

### 1 2 3 4 WORKED EXAMPLE

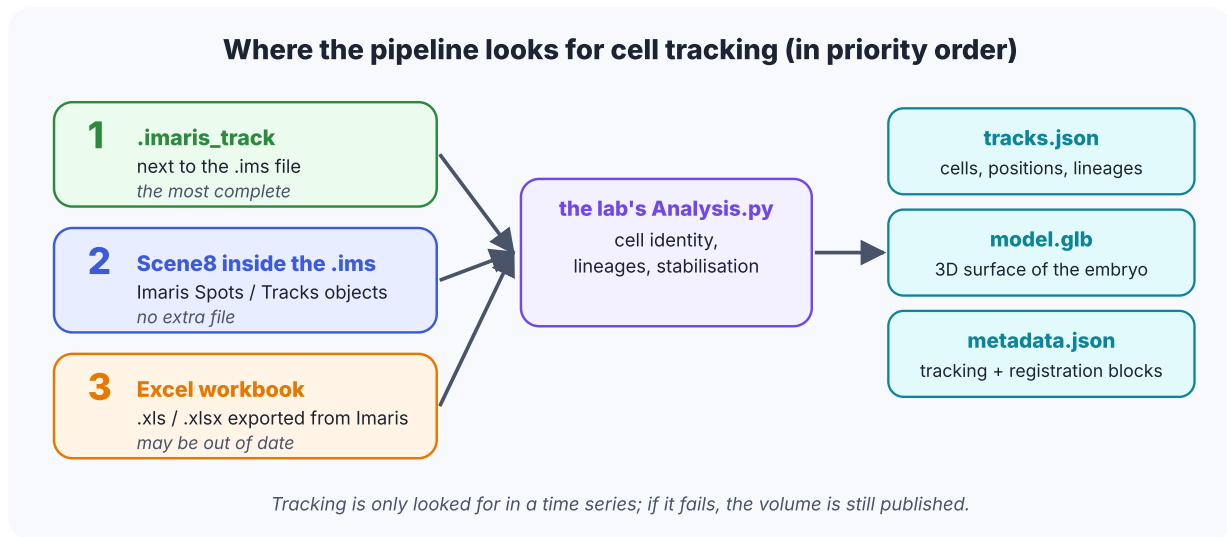
The pipeline reads the **physical extent** (ExtMin → ExtMax) and divides it by the number of voxels.  $921.6 \mu\text{m}$  over 768 voxels → **1.2  $\mu\text{m}$**  per voxel in X.  $336 \mu\text{m}$  over 112 slices → **3.0  $\mu\text{m}$**  in Z. nm and mm are converted to  $\mu\text{m}$ . A missing calibration is **never invented**.

## 2.3 Colours, channels, 8- or 16-bit data

- Imaris **colours** are not carried over: each channel gets a default colour (green, sky blue, magenta, red...), which the administrator then changes.
- **8- or 16-bit** files: both are accepted. The published image is always 8-bit (§ 4).
- More than **4 channels**: all are processed, but the viewer displays only the first 4.

## 2.4 Timelapse with cell tracking

The pipeline looks for the tracking automatically, **in this order**:



▲ The three possible sources, from the most complete to the most fragile.

### ⚠ WARNING

The Excel workbook exported from Imaris can be **stale** (measured on a lab dataset: 155 of the 172 spots). Its name must contain the interval between two frames (e.g. `30min`): the analysis reads it from the name. Prefer the *Spots/Tracks* objects saved inside the `.ims` itself.

## 3. Running the pipeline

### 🕒 IN 30 SECONDS

- Everything starts from `RUN.bat`, on a Windows PC, from the **Pipeline pack** downloaded in the *Pipeline* tab of the administration.
- The `.ims` files go into `input\`, the result comes out in `output\`.
- The dataset is built aside and appears only once **entirely** finished.

## From your computer to the site: two routes



Either way, the dataset appears in the catalogue with nothing to regenerate.

▲ From your computer to the site: drop the files, run, then copy or drag.

| RUN .BAT | MENU CHOICE | WHAT IT DOES                                                                                |
|----------|-------------|---------------------------------------------------------------------------------------------|
| [1]      |             | Process Imaris volumes ( <code>.ims</code> → <code>output\</code> , cell tracking included) |
| [2]      |             | Imaris tracking analysis (Excel → <code>tracking\OUTPUT\</code> )                           |
| [3]      |             | Import 2D photographs ( <code>.tif</code> → <code>output\2d\</code> )                       |
| [4]      |             | Attach a tracking to an already processed dataset                                           |
| [5]      |             | Check the environment only                                                                  |

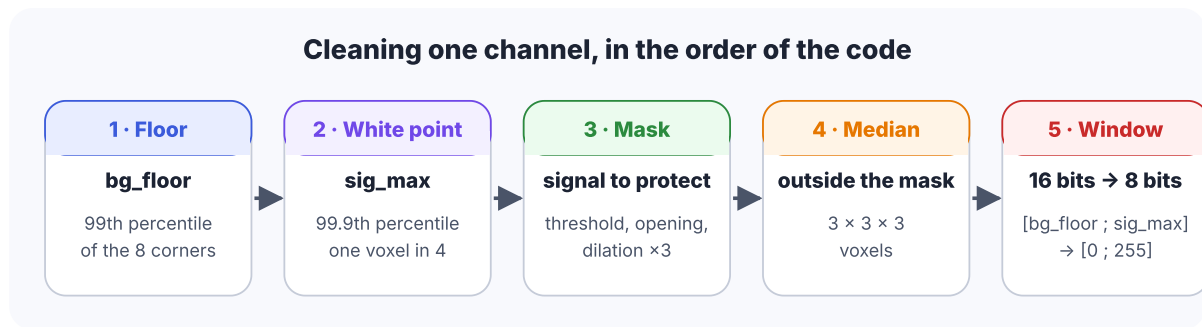
### 👉 TIP

Two editions of the pack: **light** (~3 MB, downloads Python on first run) and **full offline** (~70 MB, everything included). Library versions are pinned: two computers produce the **same bytes** from the same `.ims` . Reprocessing a dataset that is already published **keeps** your settings (name, colours, orientation, default view, gallery, visibility). → ch. 8

## 4. Cleaning the image

### ⌚ IN 30 SECONDS

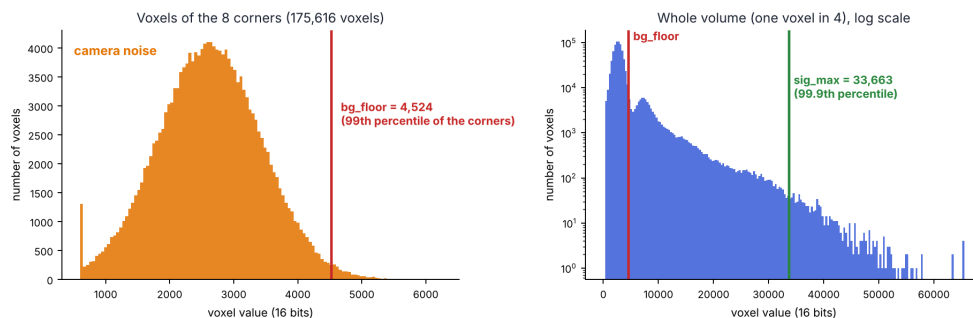
- The **camera noise** is measured in the **8 corners** of the volume, where there is no embryo.
- **Inside the signal**, nothing is changed; **outside the signal**, the noise is smoothed.
- The image is then converted to **8 bits** by a simple rule of three: the background becomes exactly **0**.



▲ The five stages of cleaning one channel.

## 4.1 Floor and ceiling

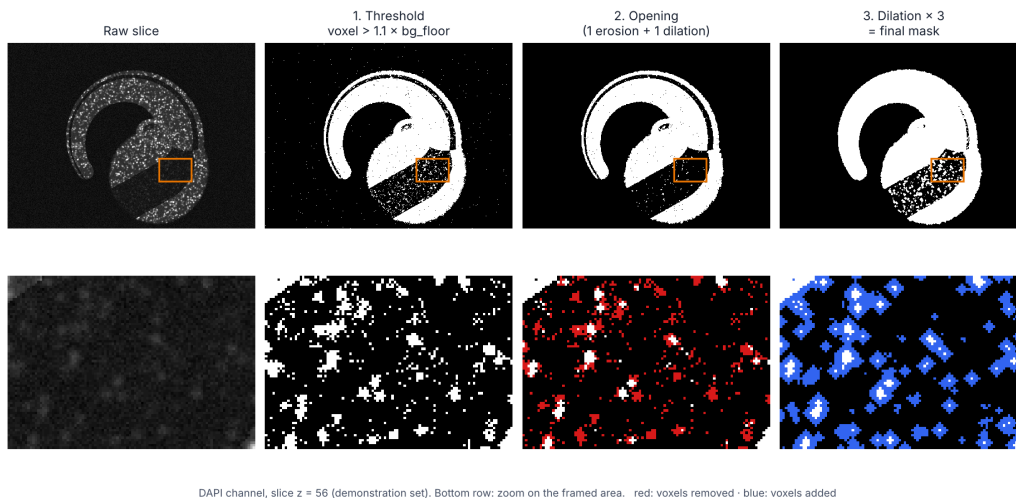
- 1 **Background floor:** 8 small cubes (up to  $32 \times 32 \times 32$  voxels) at the 8 corners. The floor = the value below which **99 %** of these corner voxels lie.
- 2 **Signal ceiling:** the value below which **99.9 %** of the voxels lie (one voxel in 4 in each direction is examined). The brightest 0.1 % saturate.



▲ Left, the corner noise forms a bell; the red line marks the 99th percentile (floor). Right, the whole volume, with the ceiling.  
Demonstration dataset, DAPI channel.

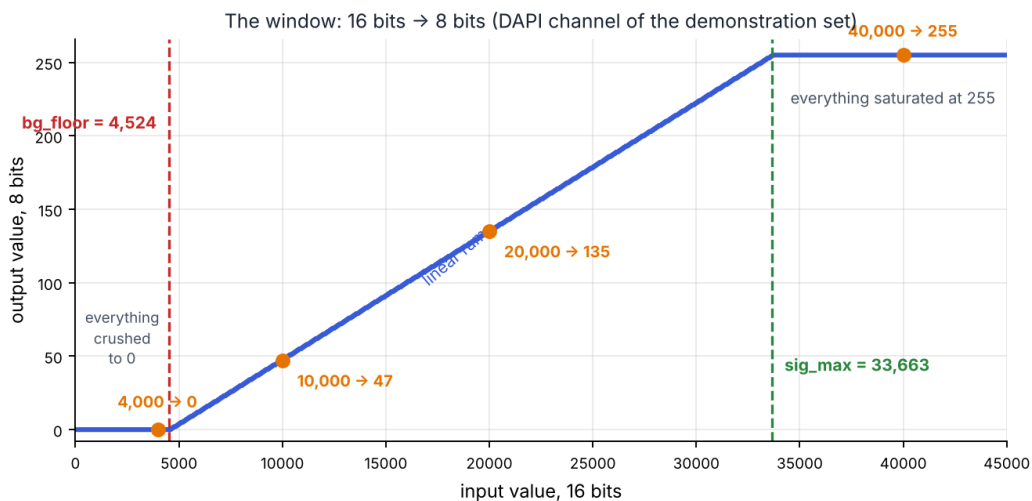
## 4.2 The mask: protecting the real signal

- 1 Every voxel **above  $1.1 \times$  the floor** is marked "signal".
- 2 An **opening** removes isolated hot pixels (a lone white dot is not a cell).
- 3 The mask is **widened by 3 voxels** to protect the natural halo around cells.
- 4 **Outside the mask only**, each voxel is replaced by the **median** of its 27 neighbours ( $3 \times 3 \times 3$ ).



▲ The mask steps on a real slice (z = 56). In red, the voxels marked “signal”. Demonstration dataset.

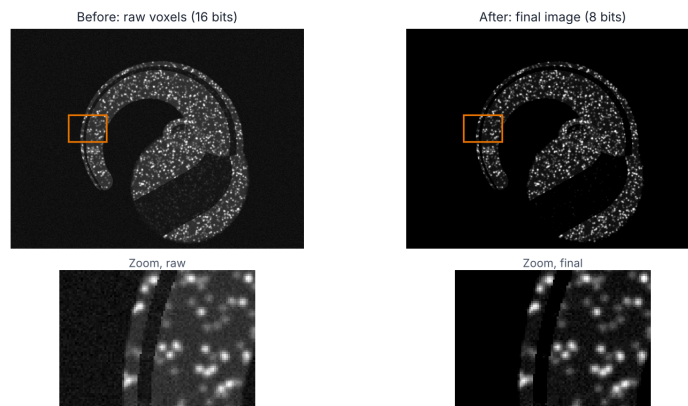
## 4.3 Conversion to 8 bits



▲ The window: 0 below the floor, 255 above the ceiling, a straight line in between (no gamma).

### WORKED EXAMPLE

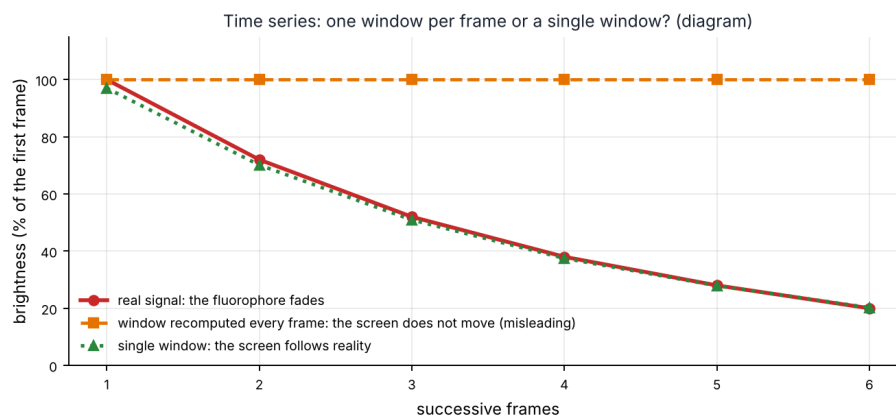
On the DAPI channel of the demonstration dataset: floor = 4,524, ceiling = 33,663. A raw voxel of **20,000** becomes  $255 \times (20,000 - 4,524) \div (33,663 - 4,524) = \mathbf{135}$  (decimal part truncated). A voxel at 4,000 becomes **0**; a voxel at 40,000 becomes **255**. Result: **88 %** of the voxels are 0.



Same slice z = 56, DAPI channel (demonstration set). The raw one is displayed from 0 to sig\_max for a fair comparison.

▲ The same slice before (raw) and after cleaning. The background disappears, the cells stay sharp.

## 4.4 For a timelapse: one shared window



▲ With one window per frame, a fading series would seem to keep the same intensity. Lumen3D uses a single window for the whole series (diagram, illustrative numbers).

The floor and ceiling are computed over **up to 8 frames** spread across the series. The signal level of each frame is **recorded** in `metadata.json`, but no viewer setting uses it yet: a fading series is really displayed darker.

### ? WHY THIS CHOICE?

An automatic thresholding method (Otsu) and a neural-network denoiser (Noise2Void) were tried and then **removed** in version 0.12.0: they created artificial coloured "blobs". → ch. 5

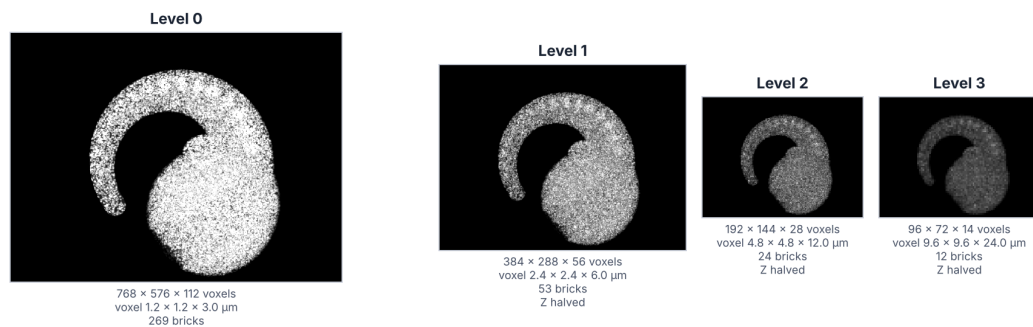
# 5. Reducing the size without loss

## ⌚ IN 30 SECONDS

- **8 bits** instead of 16: half as many bytes.
- A **pyramid** of smaller and smaller versions shows something fast, then refines it.
- The volume is cut into **bricks** of  $64^3$  voxels compressed **losslessly**; empty bricks are **not stored**.

## 5.1 The pyramid

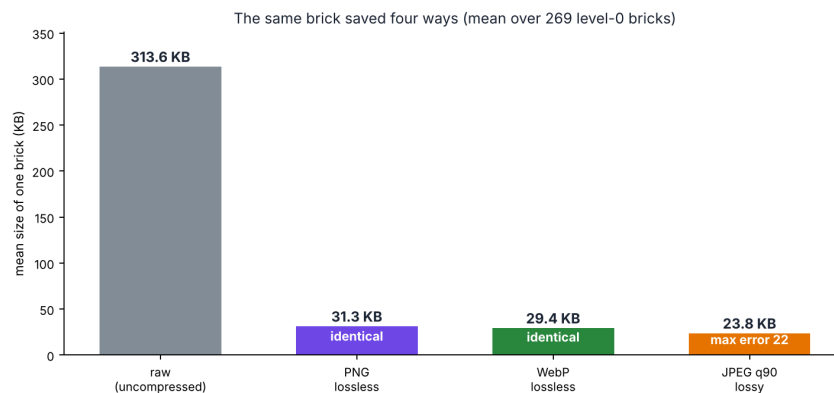
The resolution pyramid: same projection (MIP), levels 0 to 3



▲ The four levels of the demonstration dataset, to scale. Each level is the average of  $2 \times 2$  blocks ( $\times 2$  in Z when the voxels are fine enough in Z) of the level below.

- X and Y are halved at each level; **Z only as long as** the Z voxel does not become more than 1.5 times larger than in XY.
- The average is an **integer, rounded to the nearest**. The pyramid stops when the longest side is 128 voxels or less.
- Level 0 is the cleaned image as it is, **never resampled**. → ch. 6

## 5.2 Lossless compression



▲ Average size of one brick of the demonstration dataset. Only JPEG is smaller... but it changes the values.

| FORMAT                 | AVERAGE SIZE   | EXACT VALUES?                                            |
|------------------------|----------------|----------------------------------------------------------|
| Raw                    | 313.6 KB       | yes                                                      |
| PNG (best compression) | 31.3 KB        | yes                                                      |
| Lossless WebP (chosen) | <b>29.4 KB</b> | yes                                                      |
| JPEG quality 90        | 23.8 KB        | <b>no:</b> up to 22 levels off, 11.7 % of voxels changed |

### ? WHY THIS CHOICE?

These are **measurements**: lossy compression would invent signal (2.2 % of the voxels at 0 become non-zero in JPEG). WebP is a little more compact than PNG and the browser decodes it very fast. → ch. 6

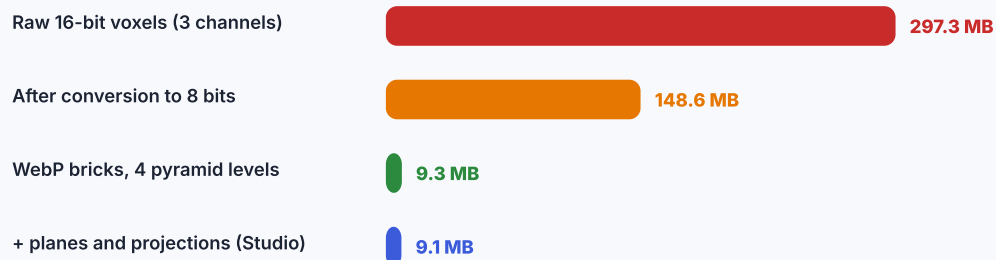
## 5.3 Empty bricks are not stored



▲ Green: stored brick. Red: brick whose voxels are all 0, not stored. Demonstration dataset, level 0.

A brick is stored only if **at least one voxel** of its interior is 1 or more. On the demonstration dataset: **358 bricks stored out of 777** possible.

### The demonstration dataset: from 297 MB of voxels to 9 MB of bricks



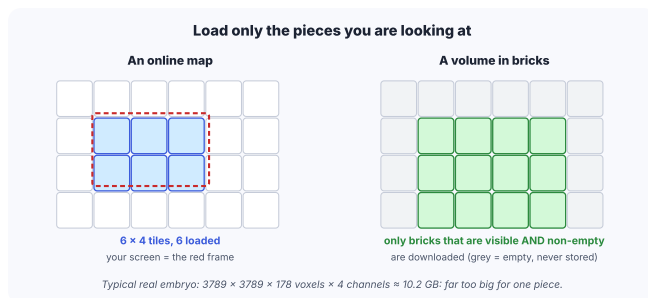
*Big win: the background becomes exactly 0, empty bricks are not stored and the rest compresses losslessly.*

▲ From the original file to the published folder: 297 MB of raw voxels become 9 MB of bricks (demonstration dataset).

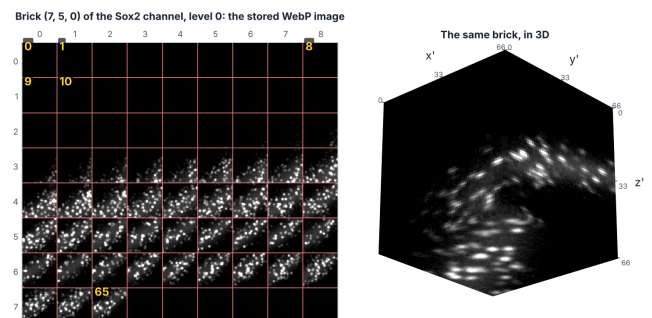
## 6. Bricks and formats

### ⌚ IN 30 SECONDS

- A brick =  **$64^3$  useful voxels** + a **one-voxel border**, laid out flat in **one** WebP image.
- Bricks are grouped into **packs**; a small **index** says where to find each one.
- **XY planes** and **per-layer projections** speed up the Studio's figures.



▲ Like the tiles of an online map: only the visible, non-empty bricks are downloaded.



▲ A real brick of the demonstration dataset: its 66 slices arranged 9 × 8 in a single image.

- **The border** copies the neighbouring voxel: the image can be smoothed with no visible seam between two bricks. → ch. 7
- **The planes** ( [planes/](#) ): a full-resolution XY slice reads **94 KB instead of 5,078 KB** on the demonstration dataset.
- **The formats**: a dataset carries a format number (1 to 4, the latest is 4). Pipeline 0.21.0 writes format 4 directly. An older dataset is upgraded from the *Data updates* tab, without reprocessing. → ch. 17

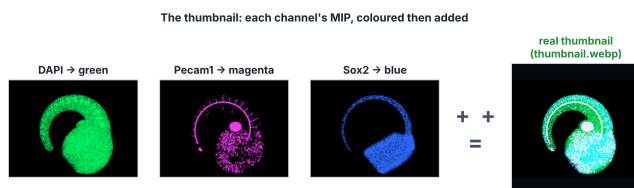
## 7. The end of the pipeline

### ⌚ IN 30 SECONDS

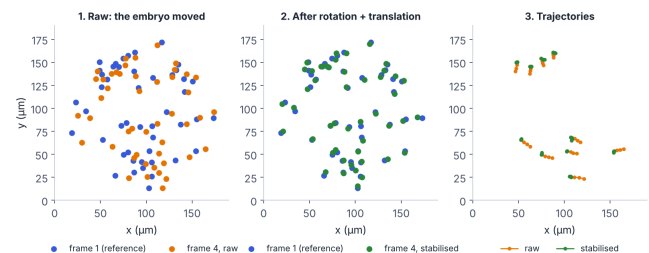
- The pipeline writes a **record** ( [metadata.json](#) ), a **thumbnail**, the channel **histograms**.
- It attaches the **cell tracking** of a timelapse and **stabilises** the positions.
- It publishes **all or nothing**: never a half-copied dataset.

| PUBLISHED FILE                 | ROLE                                                               |
|--------------------------------|--------------------------------------------------------------------|
| <a href="#">metadata.json</a>  | the record: name, stage, dimensions, calibration, channels, format |
| <a href="#">thumbnail.webp</a> | the explorer thumbnail (false-colour maximum projection)           |

| PUBLISHED FILE                                    | ROLE                                                                                |
|---------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>bricks/</code>                              | the bricks of every level and their index                                           |
| <code>planes/</code> , <code>mips/</code>         | XY planes and per-layer projections of 64 planes                                    |
| <code>tracks.json</code> , <code>model.glb</code> | cell tracking and embryo surface (timelapses)                                       |
| <code>download/</code>                            | (option) the original <code>.ims</code> , an ImageJ TIFF, PNG projections, a README |



▲ The thumbnail: three maximum projections, coloured and added.



▲ Stabilisation: a rotation + translation (no deformation) superimposes the cells of two time points.

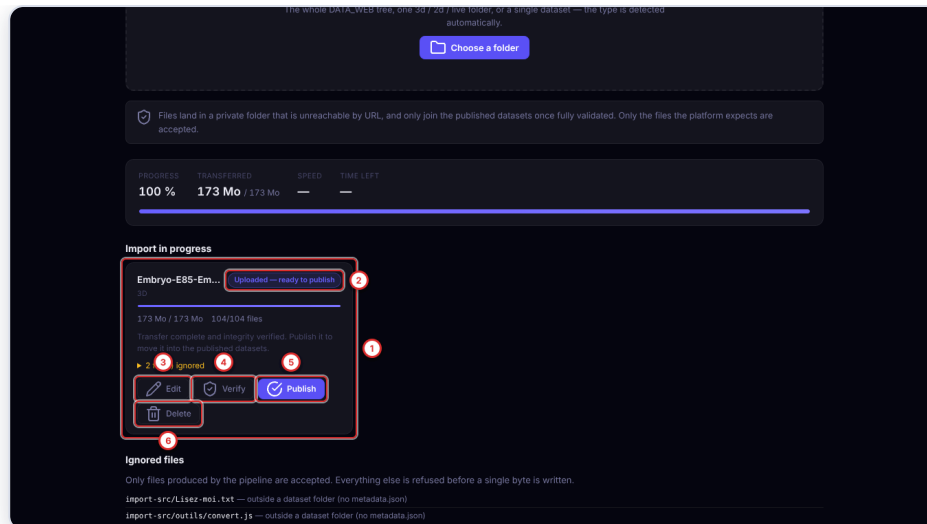
### **i** GOOD TO KNOW

**Stabilisation.** The embryo moves during the acquisition. For each time point, the pipeline computes the **rigid** displacement that best superimposes the cells (Kabsch method, at least 3 reference cells). The viewer applies this displacement **to the view**, not to the voxels: no voxel is resampled. → ch. 8 and 13

## 8. Putting the dataset online and setting it up

### **⌚** IN 30 SECONDS

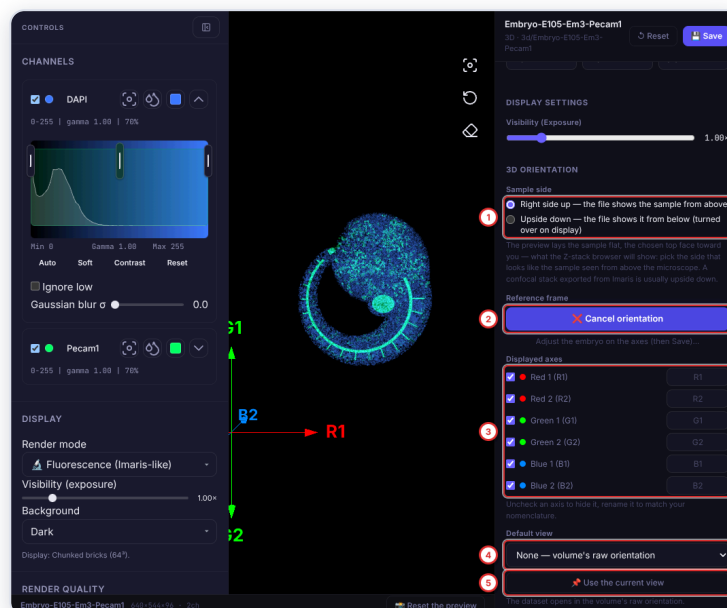
- Two routes: **copy** the folder into `DATA_WEB/` on the server, or **drag** it into the *Import* tab of the administration.
- An imported dataset is **hidden** until you show it.
- The administration sets the name, stage, colours, calibration, **orientation** and **default view**.



▲ The Import tab: a dataset fully transferred and verified, ready to publish (Edit, Verify, Publish, Delete buttons).

- The transfer **resumes by itself** after an interruption: just drag the same folder again (within 7 days).
- The upload order is designed for you: the record, the thumbnail and the **coarsest level** go first, so the dataset can be **set up within minutes**, while the rest arrives.
- Only files produced by the pipeline are accepted. → Administrator guide, ch. 4

## 8.1 The settings that matter to you



▲ Orientation in the administration: sample side, axis frame, displayed axes, default view.

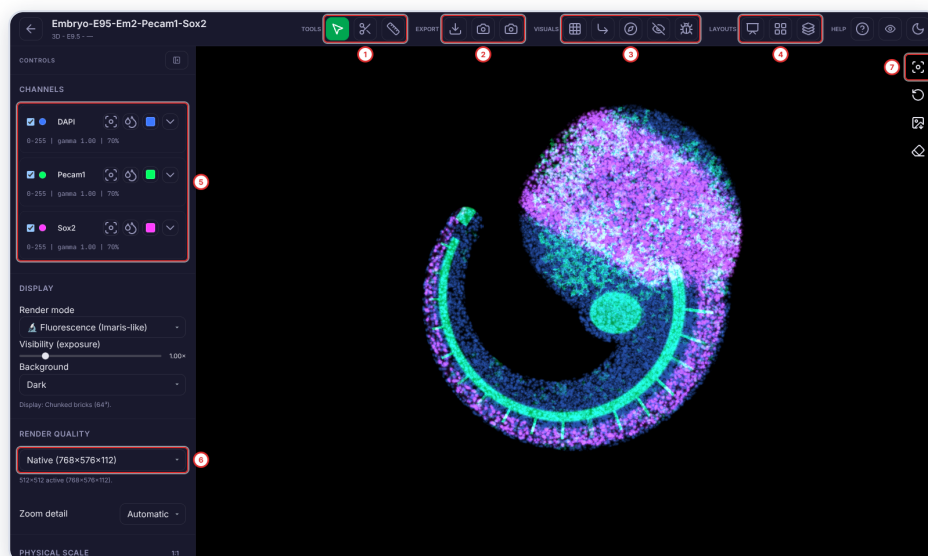
| SETTING                          | WHAT IT IS FOR                                            |
|----------------------------------|-----------------------------------------------------------|
| Name, stage, embryo, description | what visitors read in the explorer                        |
| Channel colours                  | the default colour of each channel when the dataset opens |
| Calibration                      | fix a missing or wrong voxel size                         |

| SETTING            | WHAT IT IS FOR                                                                                                        |
|--------------------|-----------------------------------------------------------------------------------------------------------------------|
| Sample side        | "Upside down" for a stack seen from below (common for an Imaris export): the Z-stack browser will show the right face |
| Define orientation | align the red / green / blue axes with the anatomy, then rename them (anterior, dorsal...)                            |
| Default view       | the pose in which the dataset opens                                                                                   |
| Gallery            | attach annotated figures, visible in the viewer                                                                       |
| Visibility         | show or hide the dataset in the public explorer                                                                       |

## 9. Exploring in 3D

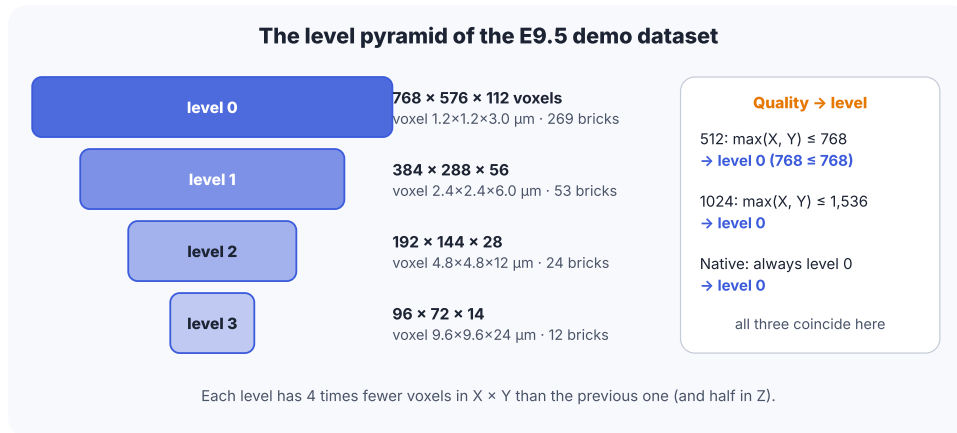
### 🕒 IN 30 SECONDS

- Open a dataset from the **explorer** (search, filters by type and stage).
- The viewer first shows a **coarse image**, then the detail arrives **starting from the centre**.
- Three **render modes** for three different questions.



▲ The viewer: toolbar at the top, channels on the left, the volume in the centre (demonstration dataset).

## 9.1 Quality and loading

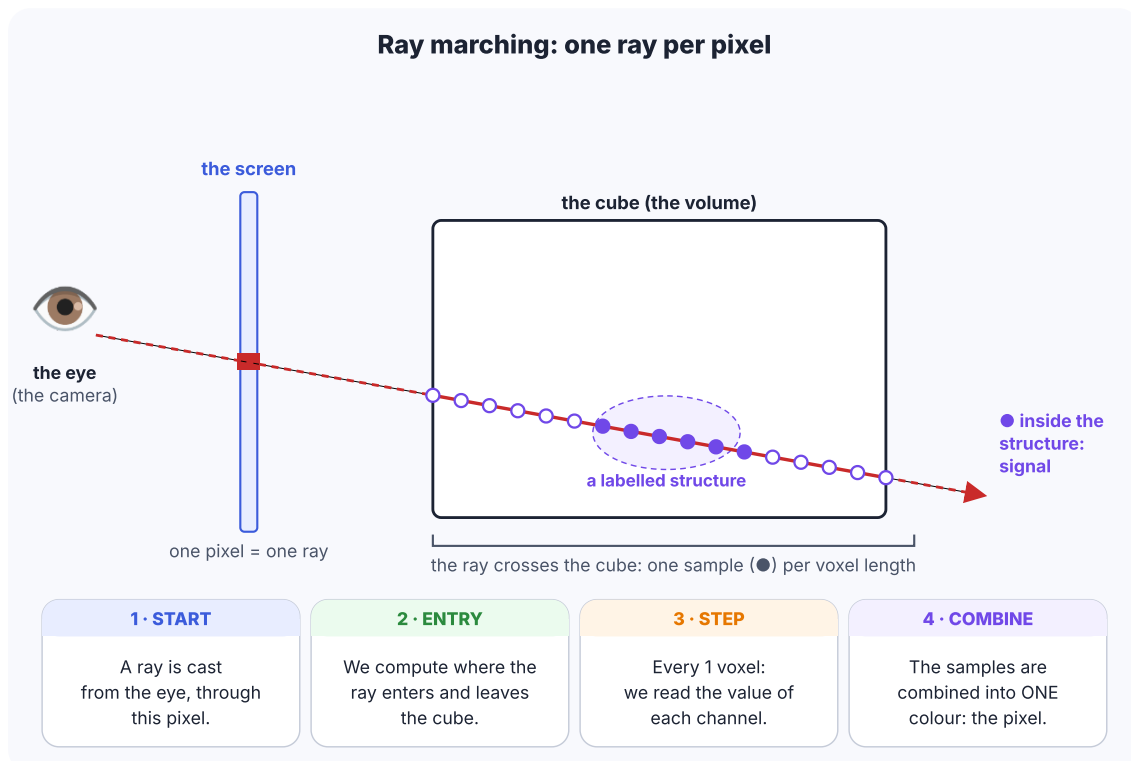


▲ The real pyramid of the demonstration dataset and the rule of the three qualities.

| QUALITY       | LEVEL LOADED                                                    |
|---------------|-----------------------------------------------------------------|
| 512 (default) | the finest level whose longest XY side is at most <b>768</b> px |
| 1024          | the finest level whose longest side is at most <b>1,536</b> px  |
| Native        | level 0, full resolution                                        |

If the graphics card does not have enough memory, the viewer takes **the next lighter level** and says so in a message. A progress line shows the quality, the percentage and the current step. → ch. 10

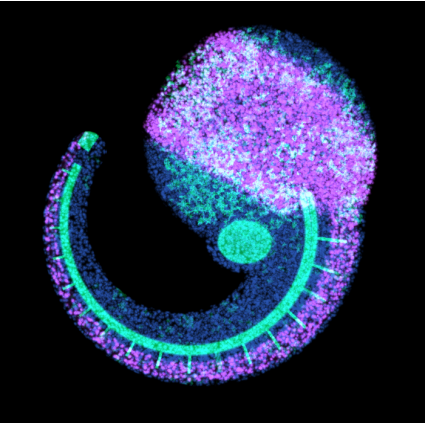
## 9.2 How the image is computed



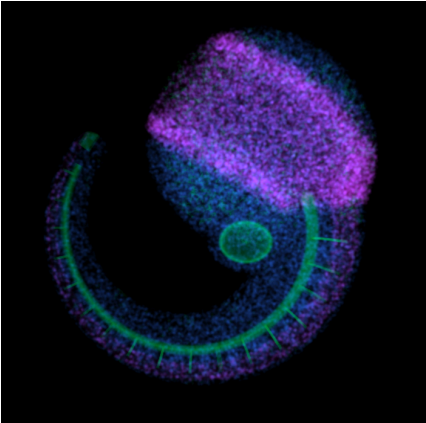
▲ One ray per pixel: it enters the volume, takes one sample per voxel crossed, and combines these values.

The image is not a photo: it is **recomputed** at every movement by the graphics card, thousands of pixels in parallel (Three.js + WebGL2). While you rotate, the definition drops a little to stay smooth; at rest, it returns to the maximum (one sample per voxel). Empty bricks are skipped. → ch. 9

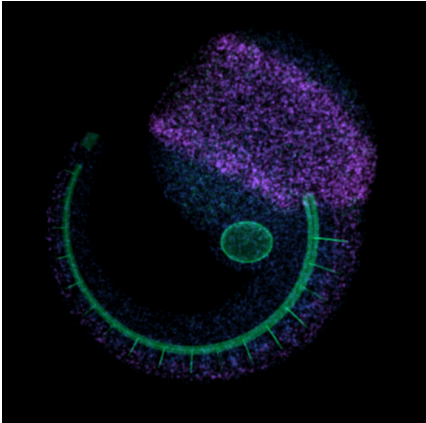
### 9.3 The three render modes



▲ Fluorescence (default).



▲ Natural fluorescence.



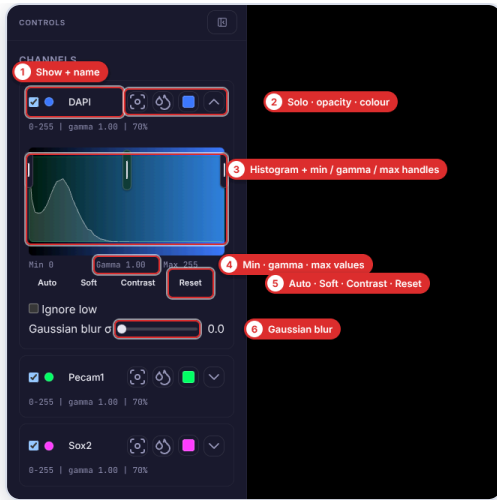
▲ Structure (DVR).

| MODE                 | WHAT YOU SEE                                                              | GOOD FOR...                       |
|----------------------|---------------------------------------------------------------------------|-----------------------------------|
| Fluorescence         | the <b>maximum</b> of each channel along the ray, colours added, no depth | seeing all the signal, like a MIP |
| Natural fluorescence | each fluorophore glows; what is dense <b>hides</b> what lies behind       | perceiving shape and depth        |
| Structure (DVR)      | an opacity accumulated from front to back                                 | surfaces and solid volumes        |

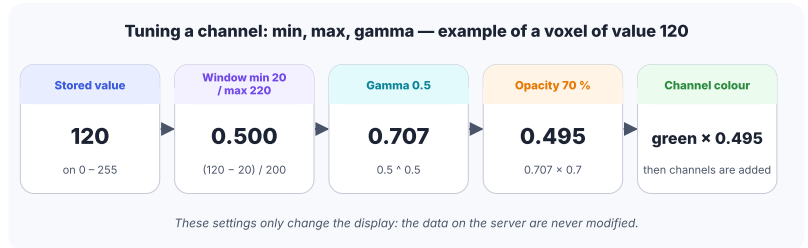
## 10. Adjusting how a channel is displayed

🕒 IN 30 SECONDS

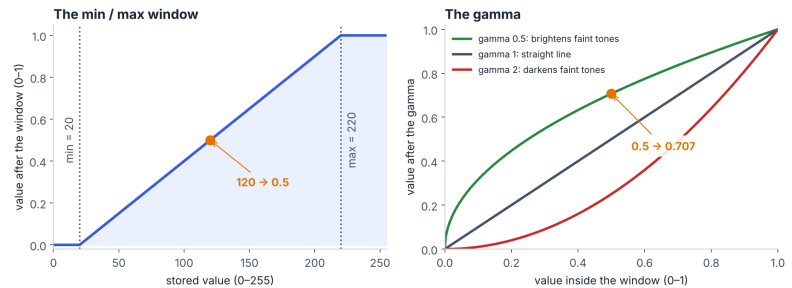
- Each channel has a **min**, a **max**, a **gamma**, an **opacity** and a **colour**; channels are **added**.
- The **histogram** shows how the 0–255 values of the coarsest level are spread, in 64 bars.
- These settings change **only the display**, never the data.



▲ A channel card, unfolded.



▲ The computation of one voxel, from storage to screen.

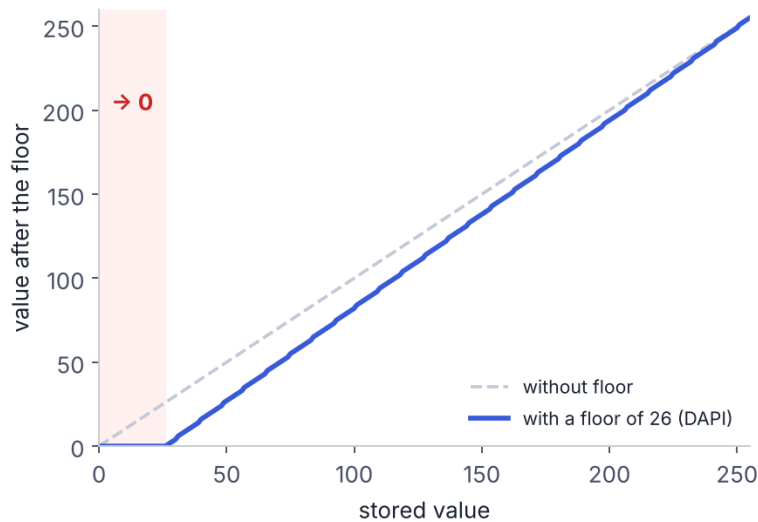


▲ Left, the window min 20 / max 220; right, the effect of gamma 0.5 / 1 / 2.

- **Min / Max:** everything below *min* becomes black, above *max* full colour.
- **Middle handle:** the level displayed at 50 %; it sets the gamma ( $\text{gamma} = \ln 0.5 \div \ln m$ ).
- **Auto** clips the darkest and brightest 0.5 %; **Soft**, **Contrast** and **Reset** are presets.
- **Opacity:** 70 %, 42 % or 100 %; **Exposure:** from  $0.2\times$  to  $5\times$  for all channels; **Solo** isolates one channel.
- **Gaussian blur:** plane-by-plane smoothing ( $\sigma$  in voxels), available at qualities that fit in a single texture.
- A **colour-blindness simulation filter** checks that a figure stays readable.

### ⚠ WARNING

**A hidden setting.** Before your settings, the viewer also sets the faintest values of each channel to 0 (a “floor” between 6 and 48, estimated from the histogram), then stretches the rest. This is why the residual noise disappears on screen. → ch. 11

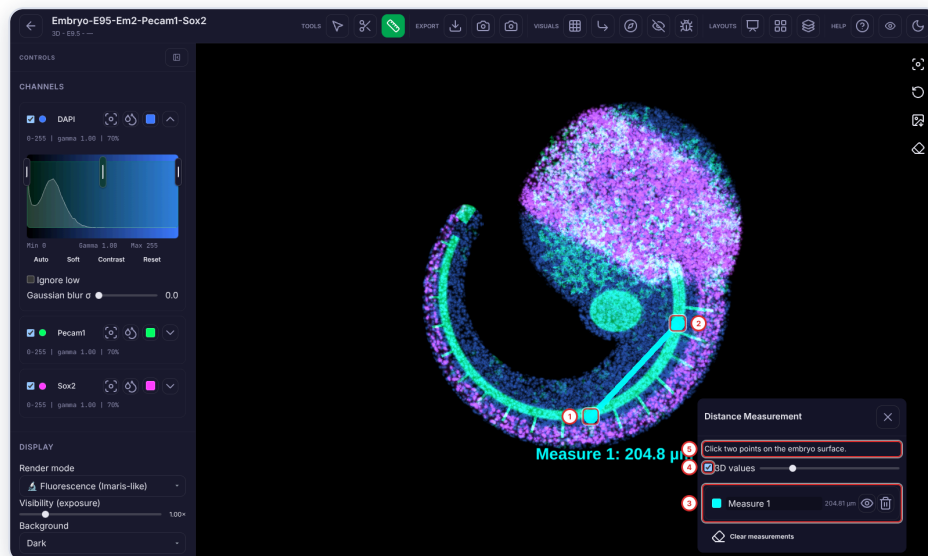


▲ The effect of the display floor: estimated at 26 for the DAPI channel of the demonstration dataset.

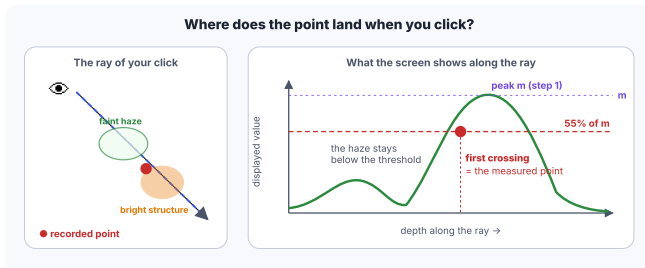
## 11. Measuring and exploring the inside

### 🕒 IN 30 SECONDS

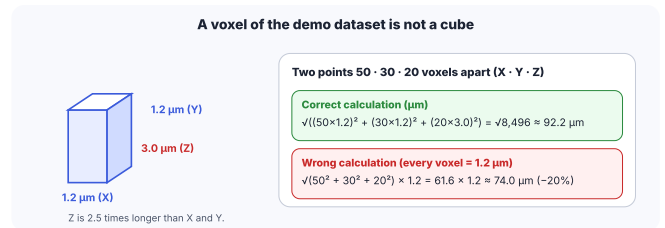
- **Measurements** are in  $\mu\text{m}$ , with the true voxel size, on the first visible structure under the click.
- **Oblique slice** and **Z-stack browser** show the inside, with an exact scale bar.
- The scale bar of the **3D view** is exact only at the depth of the sample's centre.



▲ Two points and the measured distance: 204.8  $\mu\text{m}$  (demonstration dataset).



▲ The measured point is the first structure that reaches 55 % of the displayed maximum along the ray. A click in empty space is refused.



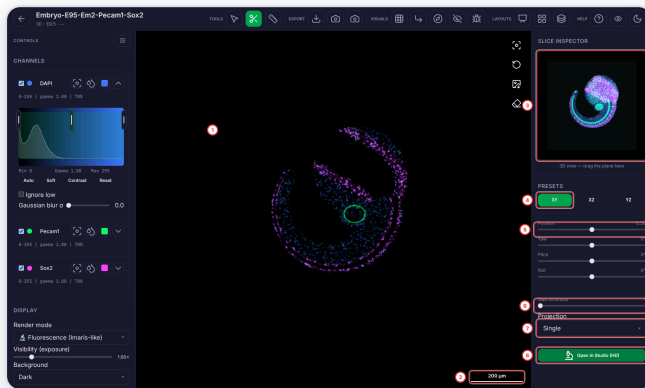
▲ Voxels are not counted: in Z, a voxel is 2.5 times longer here.

### WORKED EXAMPLE

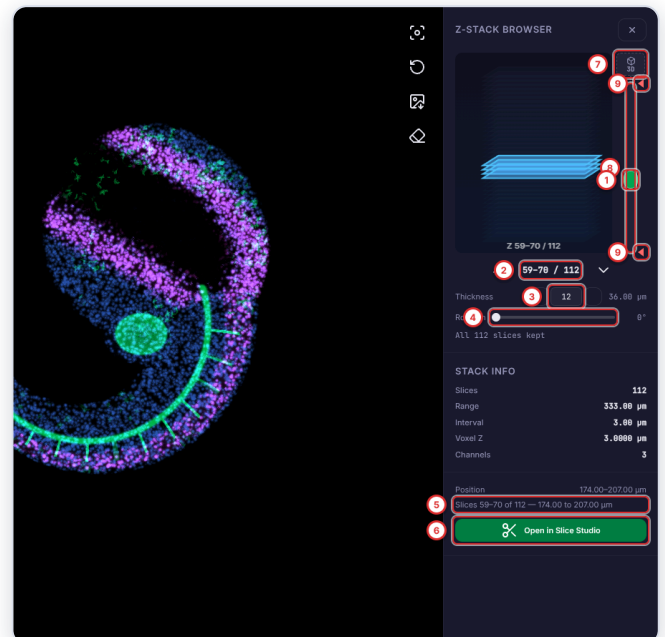
Two points 10 voxels apart in X and 4 voxels apart in Z, with voxels of  $1.2 \times 1.2 \times 3 \mu\text{m}$ :  $\sqrt{((10 \times 1.2)^2 + (4 \times 3)^2)} = \sqrt{(144 + 144)} = 17 \mu\text{m}$  — not  $\sqrt{(10^2 + 4^2)} \approx 10.8$  “voxels”.

### WARNING

**Measurements are not saved on the server:** closing the tab erases them. Keep them by copying the page address (they travel in the link) or by downloading the **measurements CSV** from the Download Center.

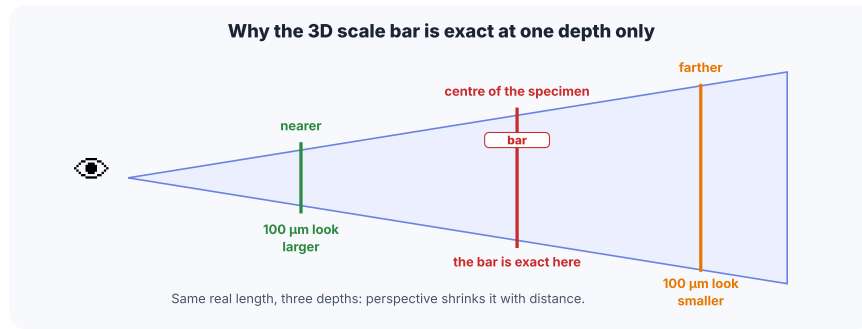


▲ XY slice through the middle of the volume, with its scale bar.



▲ Z-stack browser in slice mode: 12 slices (36 μm).

- **Oblique slice:** a plane in any direction, shown large with an exact scale bar.
- **Z-stack browser:** it lays the stack flat (top face toward you), an adjustable thickness bar, handles to trim the top and bottom, a rotation from 0 to 360°. It is exclusive with the oblique slice.
- **Orientation axes:** the red / green / blue compass (R1/R2, G1/G2, B1/B2) defined by the administrator.

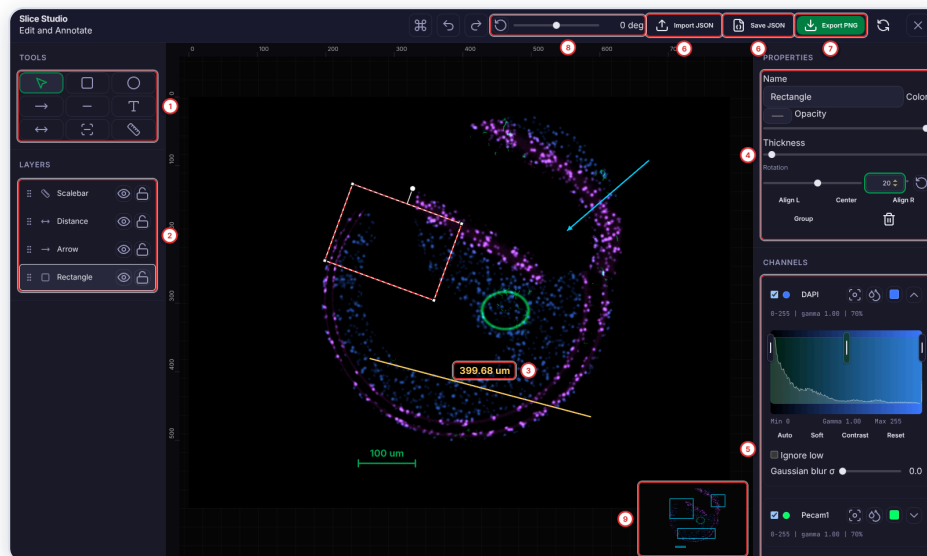


▲ Same length, three depths: the 3D view's bar is valid for the plane through the sample's centre.

## 12. Making figures and exporting

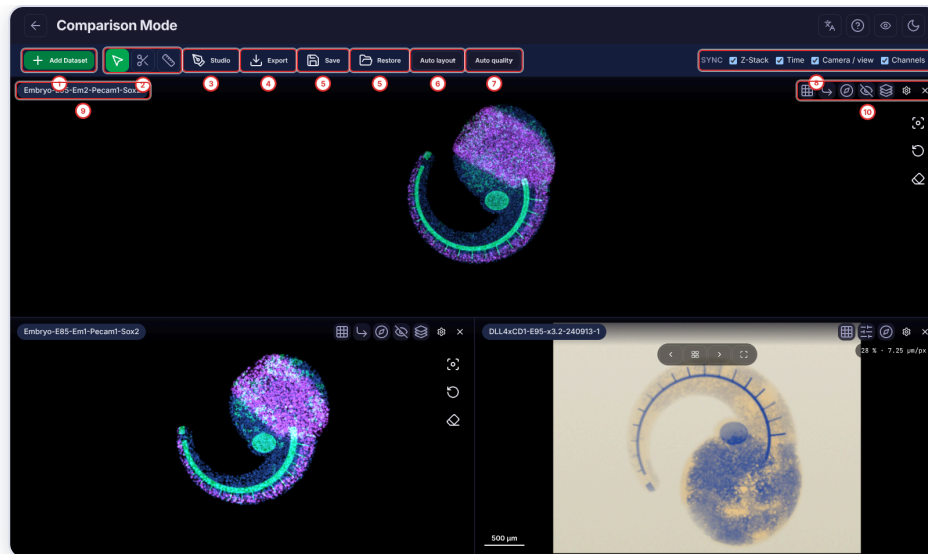
### 🕒 IN 30 SECONDS

- The **Studio** composes an annotated figure at **full resolution** (arrows, text, distances, scale bar).
- The 3D view exports **at any size**, with a transparent background if needed.
- **Compare** puts up to 4 datasets side by side, synchronised.



▲ The Studio: rotated rectangle, arrow, distance, scale bar and text, on a slice of the demonstration dataset.

- Opening a slice in the Studio reloads it **at native level** (XY planes or byte ranges), with a progress bar.
- **Channel colours** can be adjusted in the Studio without touching the annotations.
- Export as **PNG** (the figure) or **JSON** (to resume it later); above 5 MB, the export is refused with a message.



▲ Compare: two volumes and a photograph, shared tools and synchronisation (camera, time, slices, channels).

| TO...                                                   | USE                                                                              |
|---------------------------------------------------------|----------------------------------------------------------------------------------|
| get a large, sharp image of the 3D view                 | <b>Export view</b> (rendered in 2,048 px tiles, transparent background possible) |
| take a quick capture                                    | <b>Screenshot</b>                                                                |
| get the original files ( <code>.ims</code> , TIFF, MIP) | <b>Download Center</b> (if the <code>download/</code> folder exists)             |
| compare embryos or channels                             | <b>Compare</b> and <b>Decompose by channel</b>                                   |
| share exactly what you see                              | the <b>page address</b> , or the workspace saved in Compare                      |

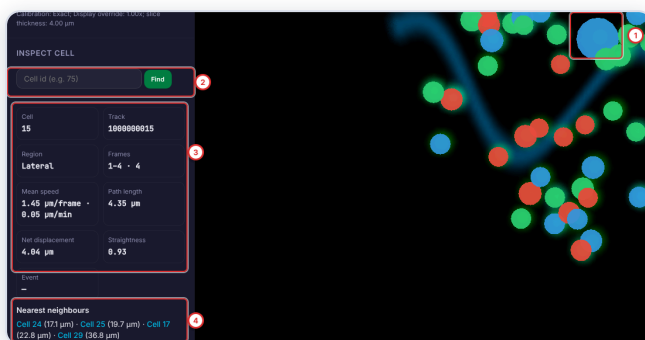
## 13. Timelapses and cell tracking

### ⌚ IN 30 SECONDS

- The **timeline** plays the series at 0.5 to 20 frames per second; a bar shows the time points already loaded.
- The **cell tracking** is overlaid on the volume: cells, trajectories, embryo surface.
- Five analysis tools: trajectories, surface, cell inspector, charts, distance between cells.



▲ A tracked timelapse (demonstration dataset, 4 time points): the tracking layer and the timeline.



▲ The inspector: metrics, lineage, neighbours of a selected cell.



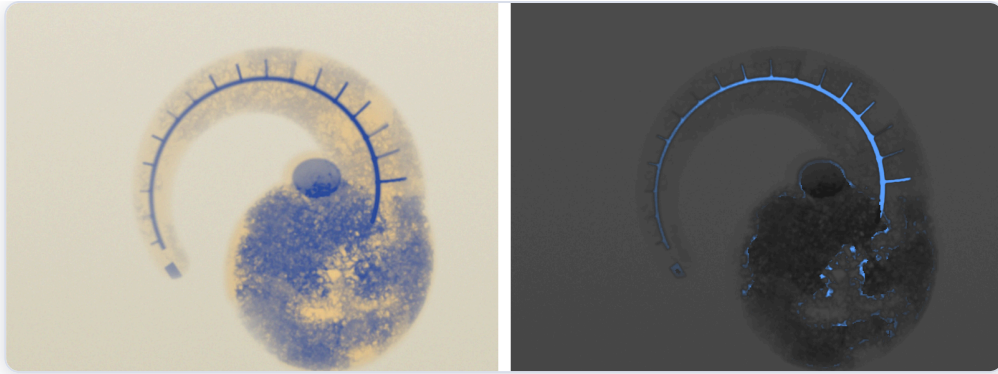
▲ The distance between two cells, followed over time.

- Playback **waits** for frames that have not arrived yet: it never silently skips a time point.
- The positions shown are the **stabilised** positions (§ 7); mitoses and fusions are marked.
- Charts (population, speed, neighbours, mitoses) and tables can be exported. → ch. 12 and 13

## 14. 2D photographs

### ⚡ IN 30 SECONDS

- One calibrated photograph per dataset: zoom, scale bar, measurements in  $\mu\text{m}$  or  $\text{mm}$ .
- **Isolate the staining** brings out the X-gal staining (blue) on the tissue (yellow).
- A contact sheet browses every photograph of the collection.



▲ Before / after “Isolate the staining”: the blue stands out, the rest turns grey (demonstration photograph).

### “Isolate the staining”: three steps for every pixel

#### 1 · Where is the yellow tissue?

a pixel's “yellowness” =  
 $(\text{red} + \text{green}) / 2 - \text{blue}$   
 It is averaged over a square  
 about 40 px wide (image  
 reduced 4 times, blur radius 5).

matte background: 2 to 6  
 broad yellow trunk: 6 to 9

#### 2 · Is the pixel blue?

ratio blue / (red + 1)  
 Unstained tissue:  $\approx 0.6$   
 Strongly stained core: 2 to 4  
**strength v: 0 if ratio  $\leq 1.0$**   
**1 if ratio  $\geq 1.8$ , linear between**

and only if the local “yellowness”  
 exceeds 5 (otherwise v = 0)

#### 3 · Repaint

grey = brightness  $\times 0.35 \times (1 - v)$

red = grey + 90  $\times v$   
 green = grey + 160  $\times v$   
 blue = grey + 255  $\times v$

v = 0: dark grey

v = 1: vivid blue (90, 160, 255)

#### Worked example

Pixel (R, G, B) = (60, 90, 170):  $170 / 61 = 2.8 \rightarrow v = 1$ , in tissue with “yellowness” 8: it turns vivid blue.  
 Pixel (200, 190, 120):  $120 / 201 = 0.6 \rightarrow v = 0$ : it turns dark grey ( $0.35 \times$  its brightness).

▲ The computation, pixel by pixel: blue / red ratio, limited to the neighbourhood of the yellow tissue to ignore background dust.

#### **i** GOOD TO KNOW

The photograph is stored as **lossy WebP (quality 90)** by default; the `--lossless` option keeps every pixel exact, useful if you want to quantify the blue / red ratio. The calibration comes from the TIFF metadata (ImageJ / Leica); without it, no scale bar. → ch. 13

# 15. Interpreting what you see correctly

## REMEMBER

- **Grey levels are relative.** Each channel of each dataset has its own window (floor → ceiling): an intensity cannot be compared between two channels or between two datasets. In a timelapse, the window is **shared by all time points**: a drop in intensity stays visible, as in reality.
- **Inside the signal, nothing is changed; outside the signal**, the noise is smoothed ( $3 \times 3 \times 3$  median).
- **8 bits** are enough for morphology; to **quantify**, go back to the original `.ims` (in [download/](#) if the option was enabled).
- **Bricks that are not stored** are areas that really are 0 after cleaning, not lost data.
- **The display floor** (6 to 48) hides the faintest values on screen: lowering *min* will not bring them back.
- **At most 4 channels** are displayed at the same time.
- **Without calibration** in the file, no measurement in  $\mu\text{m}$  is offered.
- **The 3D view is in perspective**: measure with the measuring tool or on a slice, not with the 3D scale bar.

## 15.1 What the platform does not do

| NOT AVAILABLE                     | WHY / WHAT TO DO                                            |
|-----------------------------------|-------------------------------------------------------------|
| Absolute intensity quantification | windowed 8-bit data: use the original <code>.ims</code>     |
| Segmentation, automatic counting  | do the analysis in Imaris or Fiji                           |
| Deconvolution                     | to be done before the Imaris export                         |
| Photobleaching compensation       | measured by the pipeline, but not yet applied by the viewer |
| Editing voxels                    | everything is read-only; only the display can be adjusted   |

→ ch. 20 (appendix “What Lumen3D does not do”)

## 16. What happens if... ?

| SITUATION                    | WHAT THE PLATFORM DOES                             | WHAT YOU CAN DO                 |
|------------------------------|----------------------------------------------------|---------------------------------|
| The image is blurry at first | the coarse level arrives first, the detail follows | wait for the progress to finish |

| SITUATION                          | WHAT THE PLATFORM DOES                                                               | WHAT YOU CAN DO                               |
|------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|
| "Native" shows another level       | not enough graphics memory:<br>lighter level + message                               | choose 1024, close other 3D tabs              |
| The graphics card "crashes"        | the viewer reloads the view with<br>reduced memory (at most 3 times<br>in 2 minutes) | lower the quality, reload the page            |
| A brick is corrupt                 | it is shown empty and counted,<br>never shown wrong                                  | tell the administrator                        |
| The import transfer stops          | it resumes as soon as the network<br>is back                                         | drag the same folder again if<br>needed       |
| The dataset is not in the explorer | it is hidden, or still being imported                                                | ask the administrator to publish /<br>show it |
| A measurement click is refused     | nothing visible under the cursor                                                     | click on a visible structure                  |
| The Gaussian blur is greyed out    | the chosen quality is too large for a<br>single texture                              | choose a lower quality                        |

→ ch. 19 (more than 60 situations in detail)

## 17. Words to know

### Voxel

The "pixel" of a volume: a small box, here  $1.2 \times 1.2 \times 3.0 \mu\text{m}$ . → ch. 4

### Channel

The image of one fluorophore (DAPI, Pecan1...). → ch. 4

### Floor / ceiling

The two values that define the conversion to 8 bits: below the floor → 0, above the ceiling → 255. → ch. 5

### Mask

The voxels recognised as signal; only those outside the mask are smoothed. → ch. 5

### Pyramid (levels)

The smaller and smaller versions of the volume; level 0 is full resolution. → ch. 6

### Brick

A cube of  $64^3$  voxels, the unit of cutting and downloading. → ch. 7

### Lossless WebP

The image format that stores the bricks while keeping every value exact. → ch. 6

### MIP

Maximum intensity projection: the strongest value along an axis is kept. → ch. 7 and 9

### Ray casting

The drawing method: one ray per pixel crosses the volume and combines the values it meets. → ch. 9

### Gamma

The exponent that brightens ( $< 1$ ) or darkens ( $> 1$ ) the mid-tones. → ch. 11

### Histogram

The chart that counts how many voxels have each value. → ch. 11

## Stabilisation

The rigid displacement that compensates for the embryo's drift in a timelapse. → ch. 8 and 13

## Studio

The workshop for annotated figures at full resolution. → ch. 12 and 13

## Data format

The version number of how a dataset's files are organised (1 to 4). → ch. 17

### GOING FURTHER

**Full documentation** ("Lumen3D, from A to Z"): 21 chapters, more than 350 pages, every step illustrated. **Administrator guide**: every screen of the panel, step by step. Both are published in the *Documentation* tab of the administration panel.